MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Dr. J.R.Arun kumar

Subject & Subject Code: Natural Language Processing (**6CS5-13**)

Semester: VI **Session**: 2023-24(Even Sem.)

Unit No.: II **Date of Submission**: 26/02/2024

Sl.No.	Topics Covered	Page No.	Total pages
1	Word Level Analysis: Regular Expressions, Finite-State Automata,	1-4	4
2	Morphological Parsing	5-7	3
3	Spelling Error Detection and correction, Words and Word classes	8-11	4
4	Part-of Speech Tagging.	12-14	3
5	Syntactic Analysis: Context-free Grammar, Constituency,	15-17	3
6	Parsing-Probabilistic Parsing.	18-20	3
7	Machine-readable dictionaries and lexical databases, RTN, ATN.	21-23	3

References Book:

Natural Language understanding by James Allen, Pearson Education 2008
NLP: A Paninian Perspective by Akshar Bharati, Vineet Chaitanya, and Rajeev Sangal, Prentice Hal

FACULTY HOD 27/2/2024 DEAN ACADEMICS

UNIT-2 Word Level and Syntactic Analysis

NLP - Word Level Analysis

LECTURE NO:-1

Regular Expressions, FiniteState Automata, Morphological Parsing

The language used to specify text search strings is called a regular expression (RE). Using a unique syntax that is stored in a pattern, RE aids us in matching or finding other strings or sets of strings. Both in UNIX and MS Word, regular expressions are used similarly to search text.

- **Finite State Automata:** The plural form of the word automaton is automata, and an automaton is an abstract self-propelled computing device that automatically performs a predetermined sequence of operations.
- **Morphological Parsing:** The challenge of realizing that a word can be broken down into linguistic structures known as morphemes, or smaller meaningful units, is known as morphological parsing.

Types of Morphemes: The two types of morphemes, the smallest units with meaning, are

- **Stems**: It is a core meaningful unit of a word.
- **Word order**: Morphological parsing would determine the word order. criteria for developing a morphological parser:
    - **Lexicon**: The first requirement for creating a morphological parser is a lexicon, which contains a list of stems and affixes as well as some basic details about each one. For example, whether the stem is a Noun stem or a Verb stem, and so on.
    - **Morphotactics**: It is essentially the morpheme ordering model. In other words, the model explains which morpheme classes can follow which morpheme classes within a word.
    - **Orthographic rules**: The changes that take place in a word are modeled using these spelling rules.

REGULAR EXPRESSIONS

A regular expression (RE) is a language for specifying text search strings. RE helps us to match or find other strings or sets of strings, using a specialized syntax held in a pattern. Regular expressions are used to search texts in UNIX as well as in MS WORD in identical way. We have various search engines using a number of RE features.

PROPERTIES OF REGULAR EXPRESSIONS

Followings are some of the important properties of RE −

- American Mathematician Stephen Cole Kleene formalized the Regular Expression language.
- RE is a formula in a special language, which can be used for specifying simple classes of strings, a sequence of symbols. In other words, we can say that RE is an algebraic notation for characterizing a set of strings.
- Regular expression requires two things, one is the pattern that we wish to search and other is a corpus of text from which we need to search.

Mathematically, A Regular Expression can be defined as follows −

- **ε** is a Regular Expression, which indicates that the language is having an empty string.
- **φ** is a Regular Expression which denotes that it is an empty language.
- If **X** and **Y** are Regular Expressions, then
  - **X, Y**
  - **X.Y(Concatenation of XY)**
  - **X+Y (Union of X and Y)**
  - **X*, Y* (Kleen Closure of X and Y)**

are also regular expressions.

- If a string is derived from above rules then that would also be a regular expression.

EXAMPLES OF REGULAR EXPRESSIONS

The following table shows a few examples of Regular Expressions −

Regular Expressions	**Regular Set**
(0 + 10*)	{0, 1, 10, 100, 1000, 10000, … }
(0*10*)	{1, 01, 10, 010, 0010, …}
(0 + ε)(1 + ε)	{ε, 0, 1, 01}
(a+b)*	It would be set of strings of a's and b's of any length which also includes the null string i.e. {ε, a, b, aa , ab , bb , ba, aaa…….}
(a+b)*abb	It would be set of strings of a's and b's ending with the string abb i.e. {abb, aabb, babb, aaabb, ababb, ……..}

(11)*	It would be set consisting of even number of 1's which also includes an empty string i.e. {ε, 11, 1111, 111111, …….}
(aa)*(bb)*b	It would be set of strings consisting of even number of a's followed by odd number of b's i.e. {b, aab, aabbb, aabbbbb, aaaab, aaaabbb, ……..}
(aa + ab + ba + bb)*	It would be string of a's and b's of even length that can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null i.e. {aa, ab, ba, bb, aaab, aaba, ……..}

REGULAR SETS & THEIR PROPERTIES

It may be defined as the set that represents the value of the regular expression and consists specific properties.

Properties of regular sets

- If we do the union of two regular sets then the resulting set would also be regula.
- If we do the intersection of two regular sets then the resulting set would also be regular.
- If we do the complement of regular sets, then the resulting set would also be regular.
- If we do the difference of two regular sets, then the resulting set would also be regular.
- If we do the reversal of regular sets, then the resulting set would also be regular.
- If we take the closure of regular sets, then the resulting set would also be regular.
- If we do the concatenation of two regular sets, then the resulting set would also be regular.

FINITE STATE AUTOMATA

The term automata, derived from the Greek word "αὐτόματα" meaning "self-acting", is the plural of automaton which may be defined as an abstract self-propelled computing device that follows a predetermined sequence of operations automatically.

An automaton having a finite number of states is called a Finite Automaton (FA) or Finite State automata (FSA).

Mathematically, an automaton can be represented by a 5-tuple (Q, Σ, δ, q0, F), where −

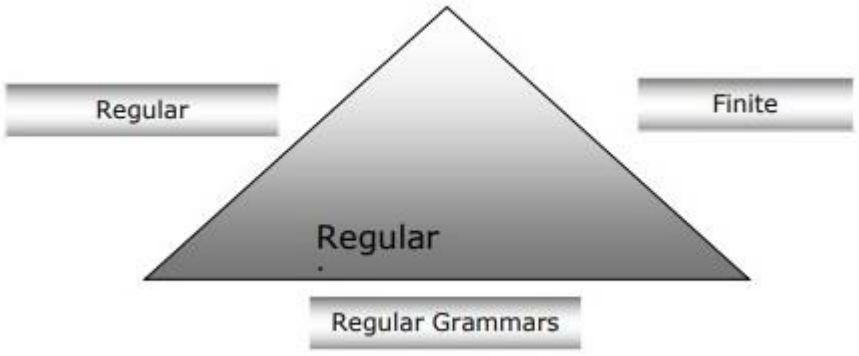
- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ is the transition function
- q0 is the initial state from where any input is processed (q0 ∈ Q).
- F is a set of final state/states of Q (F ⊆ Q).

RELATION BETWEEN FINITE AUTOMATA, REGULAR GRAMMARS AND REGULAR EXPRESSIONS

Following points will give us a clear view about the relationship between finite automata, regular grammars and regular expressions −

- As we know that finite state automata are the theoretical foundation of computational work and regular expressions is one way of describing them.
- We can say that any regular expression can be implemented as FSA and any FSA can be described with a regular expression.
- On the other hand, regular expression is a way to characterize a kind of language called regular language. Hence, we can say that regular language can be described with the help of both FSA and regular expression.
- Regular grammar, a formal grammar that can be right-regular or left-regular, is another way to characterize regular language.

Following diagram shows that finite automata, regular expressions and regular grammars are the equivalent ways of describing regular languages.



TYPES OF FINITE STATE AUTOMATION (FSA)

Finite state automation is of two types. Let us see what the types are.

Deterministic Finite automation (DFA)

It may be defined as the type of finite automation wherein, for every input symbol we can determine the state to which the machine will move. It has a finite number of states that is why the machine is called Deterministic Finite Automaton (DFA).

Mathematically, a DFA can be represented by a 5-tuple (Q, Σ, δ, q0, F), where −

- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.

- δ is the transition function where δ: Q × Σ → Q .
- q0 is the initial state from where any input is processed (q0 ∈ Q).
- F is a set of final state/states of Q (F ⊆ Q).

Whereas graphically, a DFA can be represented by diagraphs called state diagrams where −

- The states are represented by **vertices**.
- The transitions are shown by labeled **arcs**.
- The initial state is represented by an **empty incoming arc**.
- The final state is represented by **double circle**.

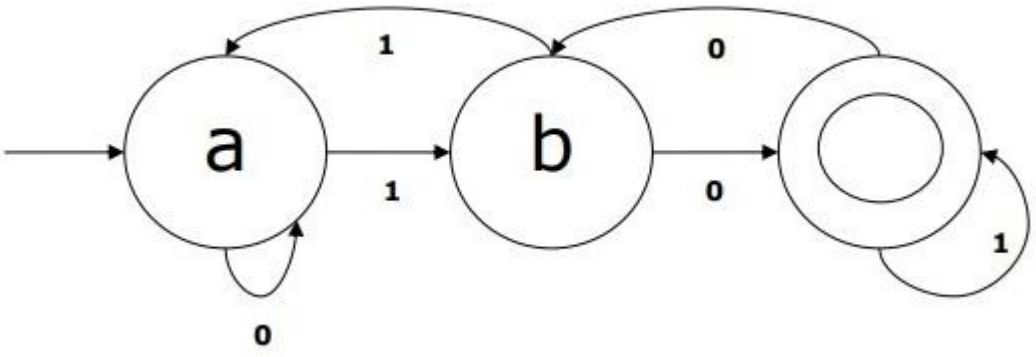
Example of DFA

Suppose a DFA be

- Q = {a, b, c},
- Σ = {0, 1},
- q_0 = {a},
- F = {c},
- Transition function δ is shown in the table as follows −

Current State	Next State for Input 0	Next State for Input 1
A	a	B
B	b	A
C	c	C

The graphical representation of this DFA would be as follows −



Non-deterministic Finite Automation (NDFA)

It may be defined as the type of finite automation where for every input symbol we cannot determine the state to which the machine will move i.e. the machine can move to any combination of the states. It has a finite number of states that is why the machine is called Non-deterministic Finite Automation (NDFA).

Mathematically, NDFA can be represented by a 5-tuple (Q, Σ, δ, q0, F), where −

- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ :-is the transition function where δ: $Q \times \Sigma \rightarrow 2^{Q}$.
- q0 :-is the initial state from where any input is processed (q0 ∈ Q).
- F :-is a set of final state/states of Q (F ⊆ Q).

Whereas graphically (same as DFA), a NDFA can be represented by diagraphs called state diagrams where −

- The states are represented by **vertices**.
- The transitions are shown by labeled **arcs**.
- The initial state is represented by an **empty incoming arc**.
- The final state is represented by double **circle**.

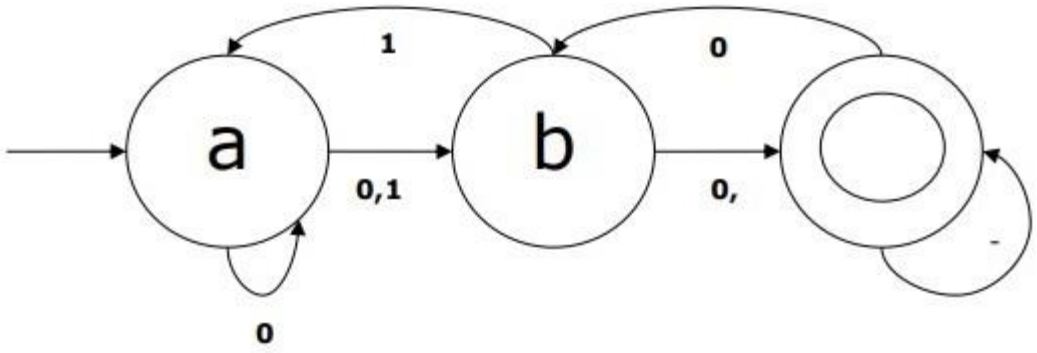
Example of NDFA

Suppose a NDFA be

- Q = {a, b, c},
- Σ = {0, 1},
- q0 = {a},
- F = {c},
- Transition function δ is shown in the table as follows −

Current State	Next State for Input 0	Next State for Input 1
A	a, b	B
B	C	a, c
C	b, c	C

The graphical representation of this NDFA would be as follows −



MORPHOLOGICAL PARSING

The term morphological parsing is related to the parsing of morphemes. We can define morphological parsing as the problem of recognizing that a word breaks down into smaller meaningful units called morphemes producing some sort of linguistic structure for it. For example, we can break the word *foxes* into two, *fox* and *-es*. We can see that the word *foxes*, is made up of two morphemes, one is *fox* and other is *-es*.

In other sense, we can say that morphology is the study of −

- The formation of words.
- The origin of the words.
- Grammatical forms of the words.
- Use of prefixes and suffixes in the formation of words.
- How parts-of-speech (PoS) of a language are formed.

TYPES OF MORPHEMES

Morphemes, the smallest meaning-bearing units, can be divided into two types −

- Stems
- Word Order

Stems

It is the core meaningful unit of a word. We can also say that it is the root of the word. For example, in the word foxes, the stem is fox.

- **Affixes** − As the name suggests, they add some additional meaning and grammatical functions to the words. For example, in the word foxes, the affix is − es.

Further, affixes can also be divided into following four types −

- **Prefixes** − As the name suggests, prefixes precede the stem. For example, in the word unbuckle, un is the prefix.
- **Suffixes** − As the name suggests, suffixes follow the stem. For example, in the word cats, -s is the suffix.
- **Infixes** − As the name suggests, infixes are inserted inside the stem. For example, the word cupful, can be pluralized as cupsful by using -s as the infix.
- **Circumfixes** − They precede and follow the stem. There are very less examples of circumfixes in English language. A very common example is 'A-ing' where we can use -A precede and -ing follows the stem.

Word Order

The order of the words would be decided by morphological parsing. Let us now see the requirements for building a morphological parser −

Lexicon

The very first requirement for building a morphological parser is lexicon, which includes the list of stems and affixes along with the basic information about them. For example, the information like whether the stem is Noun stem or Verb stem, etc.

Morphotactics

It is basically the model of morpheme ordering. In other sense, the model explaining which classes of morphemes can follow other classes of morphemes inside a word. For example, the morphotactic fact is that the English plural morpheme always follows the noun rather than preceding it.

Orthographic rules

These spelling rules are used to model the changes occurring in a word. For example, the rule of converting y to ie in word like city+s = cities not citys.

UNIT-2 Word Level and Syntactic Analysis

Word Level Analysis

LECTURE NO:-2

SPELLING ERROR DETECTION AND CORRECTION

Spelling Correction is a very important task in Natural Language Processing. It is used in various tasks like search engines, sentiment analysis, text summarization, etc. As the name suggests, we try to detect and correct spelling errors in spelling correction. In real-world NLP tasks, we often deal with data having typos, and their spelling correction comes to the rescue to improve model performance. For example, if we want to search apple and type “aple,” we will wish that the search engine suggests “apple” instead of giving no results.

DIFFERENT APPROACHES TO SPELLING CORRECTION

Norvig’s approach

In this approach, we consider all possible words that can be formed using edits - insert, delete, replace, split and transpose using the brute-force method.

For example, for word abc, all the possible words after edit are - ab ac bc bac cba acb a_bc ab_c aabc abbc acbc adbc aebc etc.

These words are then added to a list. We again repeat the same thing for the words still having errors. We then estimate all the words using the unigram language model. We pre-calculate the frequency of every word from a large corpus. The word having the highest frequency is chosen.

Adding more context

We can use n-gram models with n>1, instead of the unigram language model, providing better accuracy. But, at the same time, the model becomes heavy due to a higher number of calculations.

Increasing speed: SymSpell Approach (Symmetric Delete Spelling Correction)

Here we pre-calculate all delete typos, unlike extracting all possible edits every time we encounter any error. This will cost additional memory consumption.

Improving memory consumption

We require large datasets (at least some GBs) to obtain good accuracy. If we train the n-gram language models on small datasets like a few MBs, it leads to high memory consumption. The

symspell index and the language models occupy half-half of the size. This increased usage is because we store frequencies instead of simple text. We can compress our n-gram model by using paper. A perfect hash is a hash that can never have collisions. We can use a perfect hash to store the counts of n-grams. As we don't have any collisions, we store the count frequencies instead of the original n-grams. We need to make sure that the hash of unknown words does not match with those of known words, so for that, we use a bloom filter with known words. A bloom filter utilizes space efficiently to check whether an element belongs to a set or not. We can reduce the memory usage by using a nonlinear quantization to pack the 32-bit long count frequencies into 16-bit values.

Improving accuracy

We can use machine learning algorithms to improve accuracy further. We can start with a machine learning classifier that decides whether a given the word has an error or not. Then we can use a regression model to rank the words. This technique mimics the role of smoothing in language models as it uses all the grams as input, and then a classifier decides the rank, meaning how powerful each gram is. For word ranking, we will train a catboost(gradient boosted decision trees) ranking model with multiple features like word frequency, word length, edit distance between the source word and modified word(i.e., number of changes required to convert the source to modified), n-gram language model prediction, the frequencies of neighboring words, etc.

Further thoughts

We can gather a large corpus with errors and their corresponding corrected text and then train a model to improve accuracy.

We can also have dynamic learning options. We can learn on the way while making corrections. We can also have two pass corrections by learning some statistics in the first pass and then making the actual correction in the second pass.

UNIT-2 Word Level and Syntactic Analysis

Words and Word classes, Part-of Speech Tagging.

LECTURE NO:-3

Words and Word classes, Part-of Speech Tagging.

Word classes- 1

- Words can be grouped into classes referred to as Part of Speech (PoS) or morphological classes
  - Traditional grammar is based on few types of PoS (noun, verb, adjective, preposition, adverb, conjunction, etc..)
  - More recent models are based on a larger number of classes
    - 45 Penn Treebank
    - 87 Brown corpus
    - 146 C7 tagset
  - The word PoS provides crucial information to determine the roles of the word itself and of the words close to it in the sentence
    - knowing if a word is a personal pronoun (*I*, *you*, *he/she*,..) or a possessive pronoun (*my*, *your*, *his/her*,...) allows a more accurate selection of the most probable words that appear in its neighborhood (the syntactic rules are often based on the PoS of words)
    - e.g. *possessive pronoun - noun* vs. *personal pronoun - verb*

Word classes - 2

- Classes are usually defined on the basis of the morphological properties or of the syntactic role of words
  - words that have similar use given their affixes (morphological properties)
  - words that share similar contexts (properties related to their statistical distribution/syntactical role)
- Usually the classes are not defined on the property of semantic coherence
  - a noun is a referrer for "persons, places or things"
- The considered classes can be closed or open
  - Closed classes are those containing a fixed set of items (es. prepositions)
    - The usually contain function words (of, and, that, from, in, by,...) that are short, frequent and have a specific role in the grammar
  - Open classes are instead prone to the addition of new terms (e.g. verbs and nouns)

Word classes- nouns/names

- The 4 largest open classes of words, present in most of the languages, are
  - nouns
  - verbs
  - adverbs
  - adjectives
- Nouns are concrete terms (e.g. ship, table), abstractions (e.g. relationship, function), verb-like terms (e.g. pacing, pricing)
  - They can be functionally tied to determiners (the ship, a ship, ..) and they can assume the plural form (the ships), etc..
  - They are traditionally divided into proper nouns (e.g. Marco, Italy) and common nouns (e.g. book, lecture)
  - In many languages common nouns are also divided count nouns (they have the plural form) and mass nouns (they are used only in their singular form, e.g. snow, communism)

Word classes- verbs & adjectives

- The class of verbs includes most of the words that refer to actions and processes
  - to write, to go, to eat
  - they have "some" morphological inflections
    - In English non-3rd-person-sg (eat), 3rd-person-sg (eats), progressive (eating), past-participle (eaten), past perfect (ate)
    - A special class of verbs is that of auxiliary verbs (to be, to have)
- The class of adjectives contains terms describing properties or qualities
  - Most of the languages have adjectives for concepts like color (white, red,..), age (young, old,..) quality (good, bad,...), etc..

Word classes- adverbs

- Usually adverbs are used to modify other terms (not only verbs)
  - Directional or locative adverbs specify the direction or location of a given action (here, there, up, ..)
  - Degree adverbs specify the extent of an action, process or property (extremely, very,...)
  - Manner adverbs describe the modality of some action or process (slowly, delicately, smartly,..)
  - Temporal adverbs describe the time for an action or event (yesterday, today, before, after, later, Monday,...)
- The class of adverbs is somehow heterogeneous
  - Some adverbs are similar to nouns (e.g. Monday – we will meet Monday, we meet on Mondays)

Word classes- closed classes 1

- The closed classes are the most different among languages
  - Prepositions: from, to, on, of, with, for, by, at, ...
  - Determiners: the, a , an (il, la, lo, le, i, gli, un,..)
  - Pronouns: he, she, I, who, others,...
  - Conjunctions: and, but, or, if, because, when,...
  - Auxiliary verbs: be, have, can, must,...
  - Numerals: one, two,.., first, second
  - Particles: up, down, on, off, in, out, at, by (e.g. turn off)
- Prepositions occur before noun phrases
  - Semantically they express a relationship (spatial, temporal, etc..)
  - In English some prepositions assume a different role in predefined contexts and they are considered in the special class of particles
    - e.g. *on* in verbal phrases as "*go on*" where they have a role like an adverb

Word classes- closed classes 2

- The determiners are often at the beginning of a noun phrase
  - They are among the most common terms (e.g. the in English)
- Conjunctions are used to connect phrases, clauses or sentences
  - The coordinating conjunctions are used to join two elements at the same level (for, and, nor, but, or, yet, so are the 6 most frequent)
    - copulative (and ,also,..), disjunctive (or, nor, ..), adversative (but, however, still, yet..), illative (for, so,...), correlative (both...and, either...or, neither...nor,..)
  - Subordinating conjunctions are sued to express a fact that depends on a main clause (they define a relation between two clauses)
    - condition (unless, provided that, if, even if), reason (because, as, as if), choice (rather than, than, whether), contrast (though, although, even though, but), location (where, wherever), result/effect (in order that, so, so that, that), time (while, once, when, since, whenever, after, before, until, as soon as), concession and comparison (although, as, as though, even though, just as, though, whereas, while)

Word classes- closed classes 3

- Pronouns are short elements that are used to refer noun phrases, entities or events
  - Personal pronouns refer to persons or entities (I, you, me,..)
  - Possessive pronouns define the possess or, in general, an abstract relation between a person and an (abstract) object (my, his/her, your,..)
  - Relative pronouns are used to relate two sentences by subordinating the sentence they start with respect to the sentence containing the referred word (who, whom,...)
  - Demonstrative pronouns refer to a person or object given a spatial or temporal relation (this, that, these, those,..)
  - Indefinite pronouns are used to refer a generic object, person, event (none, nobody, everybody, someone, each one....)

Word classes- closed classes 4

- Auxiliary verbs are used in combination with other verbs to give a particular meaning to the verbal phrase (have, be, do will)
  - they are used to define the compound verb tenses (present perfect, future perfect, ..)
    - he has eaten an apple, he will go home
  - they are used to form a question or a negative form of a verb
    - *I do not (don't) walk, Do you like it?*
  - "be" is used to define the passive voice of verbs (the apple is eaten)
  - they can express a modality for the action (modal auxiliary)
    - need/requirement (must, have to, need to)
    - possibility (may)
    - will (will, wish)
    - capacity (can)

POS(PARTS-OF-SPEECH) TAGGING IN NLP

One of the core tasks in **Natural Language Processing (NLP)** is **Parts of Speech (PoS) tagging**, which is giving each word in a text a grammatical category, such as nouns, verbs, adjectives, and adverbs. Through improved comprehension of phrase structure and semantics, this technique makes it possible for machines to study and comprehend human language more accurately.

In many NLP applications, including machine translation, sentiment analysis, and information retrieval, PoS tagging is essential. PoS tagging serves as a link between language and machine understanding, enabling the creation of complex language processing systems and serving as the foundation for advanced linguistic analysis.

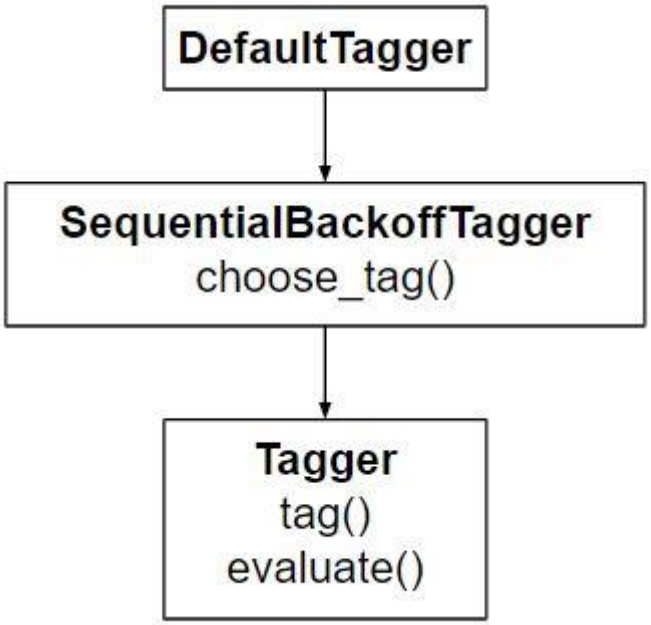
WHAT IS POS(PARTS-OF-SPEECH) TAGGING?

Parts of Speech tagging is a linguistic activity in Natural Language Processing (NLP) wherein each word in a document is given a particular part of speech (adverb, adjective, verb, etc.) or grammatical category. Through the addition of a layer of syntactic and semantic information to the words, this procedure makes it easier to comprehend the sentence's structure and meaning.

In NLP applications, POS tagging is useful for machine translation, named entity recognition, and information extraction, among other things. It also works well for clearing out ambiguity in terms with numerous meanings and revealing a sentence's grammatical structure.

Part of Speech	Tag
Noun	n
Verb	v
Adjective	a
Adverb	r

Default tagging is a basic step for the part-of-speech tagging. It is performed using the DefaultTagger class. The DefaultTagger class takes 'tag' as a single argument. **NN** is the tag for a singular noun. DefaultTagger is most useful when it gets to work with most common part-of-speech tag. that's why a noun tag is recommended.



EXAMPLE OF POS TAGGING

Consider the sentence: "The quick brown fox jumps over the lazy dog."

After performing POS Tagging:

"The" is tagged as determiner (DT)

"quick" is tagged as adjective (JJ)

"brown" is tagged as adjective (JJ)

"fox" is tagged as noun (NN)

“jumps” is tagged as verb (VBZ)

“over” is tagged as preposition (IN)

“the” is tagged as determiner (DT)

“lazy” is tagged as adjective (JJ)

“dog” is tagged as noun (NN)

By offering insights into the grammatical structure, this tagging aids machines in comprehending not just individual words but also the connections between them inside a phrase. For many NLP applications, like text summarization, sentiment analysis, and machine translation, this kind of data is essential.

Workflow of POS Tagging in NLP

The following are the processes in a typical natural language processing (NLP) example of part-of-speech (POS) tagging:

Tokenization: Divide the input text into discrete tokens, which are usually units of words or subwords. The first stage in NLP tasks is tokenization.

Loading Language Models: To utilize a library such as NLTK or SpaCy, be sure to load the relevant language model. These models offer a foundation for comprehending a language’s grammatical structure since they have been trained on a vast amount of linguistic data.

Text Processing: If required, preprocess the text to handle special characters, convert it to lowercase, or eliminate superfluous information. Correct PoS labeling is aided by clear text.

Linguistic Analysis: To determine the text’s grammatical structure, use linguistic analysis. This entails understanding each word’s purpose inside the sentence, including whether it is an adjective, verb, noun, or other.

Part-of-Speech Tagging: To determine the text’s grammatical structure, use linguistic analysis. This entails understanding each word’s purpose inside the sentence, including whether it is an adjective, verb, noun, or other.

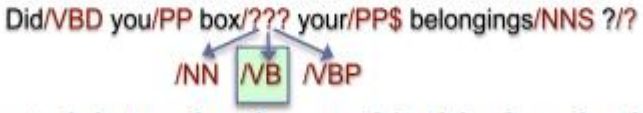
Results Analysis: Verify the accuracy and consistency of the PoS tagging findings with the source text. Determine and correct any possible problems or mistagging.

PoS tagging algorithms

- Rule-based taggers
  - The are based on the "handcrafting" of a large rule-base that specifies the conditions to be verified to assign a specific tag in the ambiguous cases
    - e.g. a word is a noun if it is preceded by a determiner
- Probabilistic taggers
  - They revolve ambiguities by estimating the probability that a given word as a specific tag in the observed context. The parameters for the probability model are estimated on a reference corpus.
- Other approaches
  - tagging can be cast as a classification task (each tag corresponds to a class and the classifiers exploits features that describe the context – e.g. features of the words on the left/right of the considered word)
  - Taggers can exploits rules learnt from examples

PoS tagging & tags

- PoS tagging consists in assigning a tag to each word in a document
  - The selection of the employed tagset depends on the language and specific application
  - The input is a word sequence and the employed tagset while the output is the association of each word to its "*best*" tag
  - There may exist more tags for a given word (ambiguity)



  - The PoS tagger task is to solve these ambiguities by selecting the most appropriate tag given the word context
    - The percentage of ambiguous words is not too high, but among them there are very frequent words (e.g. *can* – Auxiliary verb, Noun, Verb, *still* has 7 compatible tags – adj, adv, verb, noun)

Rule-based PoS tagging

- Two step process (e.g. ENGTWOL Voutilainen, 1995)
  - Word tagging using a lexicon (more than one tag can be assigned to each word) exploiting morphological/orthographic rules

text	stem	PoS	PoS features
Pavlov	**PAVLOV**	**N**	**NOM SG PROPER**
had	**HAVE** HAVE	**V** PCP2	**PAST VFIN SVO** SVO
shown	**SHOW**	**PCP2**	**SVOO SVO SV**
that	THAT THAT THAT **THAT**	ADV PRON DET **CS**	 DEM SG CENTRAL DEM SG
salivation	**SALIVATION**	**N**	**NOM SG**

N: Noun
V: Verb
PCP2: Past Participle
ADV: Adverb
PRON: Pronoun
DET: Determiner
CS: Subordinating Conjunction

SVOO: Subject-Verb-Object-Object
NOM: non-genitive
VFIN: finite verb
DEM: demonstrative

  - Application of rules to select only one tag among those assigned to each word (rules exploit the word context)

Rules

- Rules are aimed to removing the cases that are not compatible with the context
  - In ENGTWOL there are about 1100 rules

```
ADVERBIAL-THAT RULE
input: "that"
if
  (+1 A/ADV/QUANT); // the following word is an adjective, adverb orquantifier
  (+2 SENT-LIM);    // and the following one is a sentence boundary
  (NOT -1 SVOC/A);  // and the preceding word is a verb like "consider"
                    // that admits an adjective as object (I consider that good)
then eliminate non-ADV tags
else eliminate ADV tag
```

  - ENGTWOL has also probabilistic constraints and it may exploit syntactic information

Probabilistic tagging with HMM

- Given a word sequence an HMM-based tagger computes the tag sequence maximizing its probability
  - The probability is assigned to the whole tag sequence T
    - it is the tag sequence yielding the maximum likelihood given the observed word sequence W (Viterbi)

$$\hat{T} = \text{argmax}_T p(T|W) \quad T = t_1 t_2 \dots t_n \quad W = w_1 w_2 \dots w_n$$

  - By the Bayes rule the previous expression can be rewritten as

$$\hat{T} = \text{argmax}_T \frac{p(T)p(W|T)}{P(W)} = \text{argmax}_T p(T)p(W|T)$$

HMM tagging - model assumptions

- From the chain rule for probability factorization

$$p(T)p(W|T) = \prod_{i=1}^{n} p(w_i|w_1 t_1 \dots w_{i-1} t_{i-1} t_i) p(t_i|w_1 t_1 \dots w_{i-1} t_{i-1})$$

- Some approximation are introduced to simplify the model, such as
  - The word probability depends only on the tag

$$p(w_i|w_1 t_1 \dots w_{i-1} t_{i-1} t_i) = p(w_i|t_i)$$

  - The dependence of a tag from the preceding tag history is limited in time, f.i. a tag depends only on the two preceding ones

$$p(t_i|w_1 t_1 \dots w_{i-1} t_{i-1}) = p(t_i|t_{i-2} t_{i-1})$$

Q.1 What is POS tagging? Explain types of word classes in English NLP. Also comment on possible tag sets available in ENGLISH NLP. Show how the tags are assigned to the words of following sentence: “Time flies like an arrow.”

A Part-Of-Speech Tagger (POS Tagger) is a piece of software that reads text in some language and assigns parts of speech to each word (and other token), such as noun, verb, adjective, etc., although generally computational applications use more fine-grained POS tags like 'noun-plural'.

There are two broad categories of word classes:

1. Open Class

• Nouns: Proper Noun, Common Noun

• Verb

• Adjective

• Adverbs: Directional, Degree, Manner, Temporal

2. Closed Class

• Prepositions

• Determiners

• Pronouns

• Conjunctions

• Auxiliary verb

• Particles

• Numerals

In the sentence “Time flies like an arrow.”, the tags assigned will be:

Time -> Noun, Flies -> verb, Like -> Adjective, An -> Particle, Arrow -> Noun

Q.2 Why POS tagging is hard? Discuss possible challenges faced while performing POS tagging.

The problem of part-of-speech tagging seems fairly straight-forward.

For a given sentence, identify the part of speech of the various sentences.

Example: She loves small animals.

Tags: She(pronoun) loves(noun) small(adjective) animals(noun).

In some other cases, this may be difficult.

Example: This can is difficult to open.

While can is usually (auxiliary) verb but here it refers to can (a container) and is thus a noun.

Challenges of POS Tagging

1. Multiple tags and multiple words

2. Unknown words

3. Ambiguity

Q.3 Discuss various approaches to perform POS tagging.

• Rule-based POS Tagging

• Stochastic POS Tagging

• Transformation-based Tagging

• HMM POS-Tagging

Q.4 Explain in detail Rule based POS tagging/ Stochastic (HMM) POS tagging/ Hybrid POS

TAGGING.

RULE-BASED POS TAGGING

Rule-based taggers use dictionary or lexicon for getting possible tags for tagging each word. If the word has more than one possible tag, then rule-based taggers use hand-written rules to identify the correct tag. Disambiguation can also be performed in rule-based tagging by analysing the linguistic features of a word along with its preceding as well as following words. For example, suppose if the preceding word of a word is article then word must be a noun.

STOCHASTIC POS TAGGING

Another technique of tagging is Stochastic POS Tagging. Now, the question that arises here is which model can be stochastic. The model that includes frequency or probability (statistics) can be called stochastic. Any number of different approaches to the problem of part-of-speech

tagging can be referred to as stochastic tagger.

Two major ways in which tagging can be done here is:

• Word Frequency Approach

• Tag Sequence Probabilities

Q.5 Explain use of CFG in Natural Language Processing with suitable example.

A context-free grammar (CFG) is a list of rules that define the set of all well-formed sentences in a language. Each rule has a left-hand side, which identifies a syntactic category, and a right-hand side, which defines its alternative component parts, reading from left to right.

E.g., the rule s --> np vp means that "a sentence is defined as a noun phrase followed by a verb phrase." Figure 1 shows a simple CFG that describes the sentences from a small subset of English.

A sentence in the language defined by a CFG is a series of words that can be derived by systematically applying the rules, beginning with a rule that has s on its left-hand side. A parse of the sentence is a series of rule applications in which a syntactic category is replaced by the right-hand side of a rule that has that category on its left-hand side, and the final rule application yields the sentence itself. E.g., a parse of the sentence "the giraffe dreams" is:

s => np vp => det n vp => the n vp => the giraffe vp => the giraffe iv => the giraffe dreams

Figure 1. A grammar and a parse tree for "the giraffe dreams".

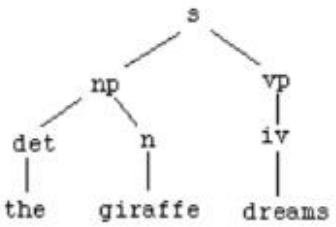
```
s   → np vp

np  → det n
vp  → tv np
    → iv

det → the
    → a
    → an

n   → giraffe
    → apple

iv  → dreams
tv  → eats
    → dreams
```



A convenient way to describe a parse is to show its parse tree, which is simply a graphical display of the parse. Figure 1 shows a parse tree for the sentence "the giraffe dreams". Note that the root of every subtree has a grammatical category that appears on the left-hand side of a rule, and the children of that root are identical to the elements on the right-hand side of that rule.

Q. 6 What is parsing? Explain/Compare Top-down & Bottom-up approach of parsing with suitable example.

PARSING: Parsing, syntax analysis, or syntactic analysis is the process of analyzing a string of symbols, either in natural language, computer languages or data structures, conforming to the rules of a formal grammar. The term parsing comes from Latin pars (orationis), meaning part (of speech).

Sr.No	Top-Down	Bottom-Up
1	It is a parsing strategy that first looks at the highest level of the parse tree and works down the parse tree by using the rules of grammar.	It is a parsing strategy that first looks at the lowest level of the parse tree and works up the parse tree by using the rules of grammar.
2	Top-down parsing attempts to find the left most derivations for an input string.	Bottom-up parsing can be defined as an attempts to reduce the input string to start symbol of a grammar.
3	In this parsing technique we start parsing from top (start symbol of parse tree) to down (the leaf node of parse tree) in top-down manner.	In this parsing technique we start parsing from bottom (leaf node of parse tree) to up (the start symbol of parse tree) in bottom-up manner.
4	This parsing technique uses Left Most Derivation.	This parsing technique uses Right Most Derivation.

Q.7 W.r.t following Grammar show Shift reduce parsing of following sentences

(1) Book that flight. (2) Does that flight include meal

S → NP VP S → Aux NP VP S → VP NP → Det NOM

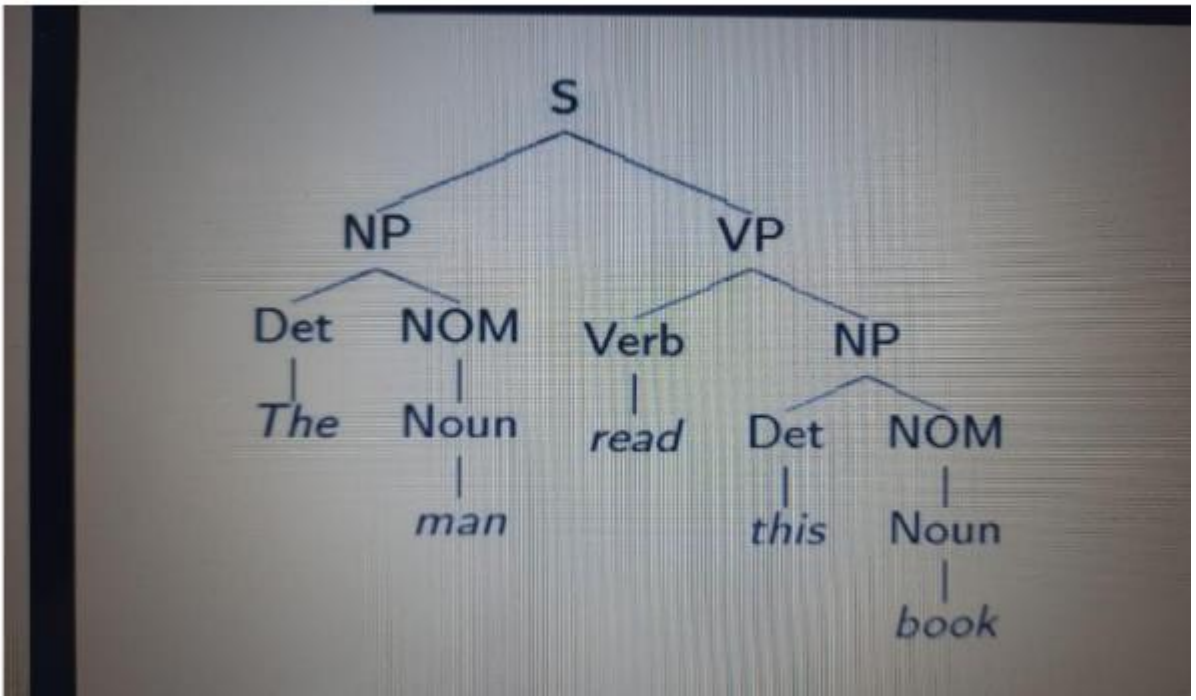
NOM → Noun NOM → Noun NOM VP → Verb

VP → Verb NP

Det → that | this | a | the Noun → book | flight | meal | man

Verb → book | include | read Aux → does

Q.8 Construct a parse tree for the following sentence using CFG rules: “The man read this book”


S
NP
VP
Det
NOM
Verb
NP
The
Noun
read
Det
NOM
man
this
Noun
book

.

UNIT-2 Word Level and Syntactic Analysis

Word Level Analysis - Context-free Grammar, Constituency, Parsing-Probabilistic Parsing

LECTURE NO:-4

WHAT IS GRAMMAR?

Grammar is defined as the rules for forming well-structured sentences. Grammar also plays an essential role in describing the syntactic structure of well-formed programs, like denoting the syntactical rules used for conversation in natural languages.

- In the theory of formal languages, grammar is also applicable in Computer Science, mainly in programming languages and data structures. Example - In the C programming language, the precise grammar rules state how functions are made with the help of lists and statements.
- Mathematically, a grammar G can be written as a 4-tuple (N, T, S, P) where:
  - **N or VN** = set of non-terminal symbols or variables.
  - **T or** ∑ = set of terminal symbols.
  - **S** = Start symbol where S ∈ N
  - **P** = Production rules for Terminals as well as Non-terminals.
  - It has the form $\alpha \rightarrow \beta$, where α and β are strings on $(V_N \cup \sum)$, and at least one symbol of α belongs to VN

Syntax

Each natural language has an underlying structure usually referred to under Syntax. The fundamental idea of syntax is that words group together to form the constituents like groups of words or phrases which behave as a single unit. These constituents can combine to form bigger constituents and, eventually, sentences.

- Syntax describes the regularity and productivity of a language making explicit the structure of sentences, and the goal of syntactic analysis or parsing is to detect if a sentence is correct and provide a syntactic structure of a sentence.

Syntax also refers to the way words are arranged together. Let us see some basic ideas related to syntax:

- **Constituency:** Groups of words may behave as a single unit or phrase - A constituent, for example, like a Noun phrase.

- **Grammatical relations:** These are the formalization of ideas from traditional grammar. Examples include - subjects and objects.
- **Subcategorization and dependency relations:** These are the relations between words and phrases, for example, a Verb followed by an infinitive verb.
- **Regular languages and part of speech:** Refers to the way words are arranged together but cannot support easily. Examples are Constituency, Grammatical relations, and Subcategorization and dependency relations.
- **Syntactic categories and their common denotations in NLP:** np - noun phrase, vp - verb phrase, s - sentence, det - determiner (article), n - noun, tv - transitive verb (takes an object), iv - intransitive verb, prep - preposition, pp - prepositional phrase, adj - adjective

TYPES OF GRAMMAR IN NLP

Let us move on to discuss the types of grammar in NLP. We will cover three types of grammar: context-free, constituency, and dependency.

Context Free Grammar

Context-free grammar consists of a set of rules expressing how symbols of the language can be grouped and ordered together and a lexicon of words and symbols.

- One example rule is to express an NP (or noun phrase) that can be composed of either a ProperNoun or a determiner (Det) followed by a Nominal, a Nominal in turn can consist of one or more Nouns: NP → DetNominal, NP → ProperNoun; Nominal → Noun | NominalNoun
- Context-free rules can also be hierarchically embedded, so we can combine the previous rules with others, like the following, that express facts about the lexicon: Det → a Det → the Noun → flight
- Context-free grammar is a formalism power enough to represent complex relations and can be efficiently implemented. Context-free grammar is integrated into many language applications
- A Context free grammar consists of a set of rules or productions, each expressing the ways the symbols of the language can be grouped, and a lexicon of words

Context-free grammar (CFG) can also be seen as the list of rules that define the set of all well-formed sentences in a language. Each rule has a left-hand side that identifies a syntactic category and a right-hand side that defines its alternative parts reading from left to right.
- **Example:** The rule s --> np vp means that "a sentence is defined as a noun phrase followed by a verb phrase."

- **Formalism in rules for context-free grammar:** A sentence in the language defined by a CFG is a series of words that can be derived by systematically applying the rules, beginning with a rule that has s on its left-hand side.

  - Use of parse tree in context-free grammar: A convenient way to describe a parse is to show its parse tree, simply a graphical display of the parse.
  - A parse of the sentence is a series of rule applications in which a syntactic category is replaced by the right-hand side of a rule that has that category on its left-hand side, and the final rule application yields the sentence itself.
- **Example:** A parse of the sentence "the giraffe dreams" is: s => np vp => det n vp => the n vp => the giraffe vp => the giraffe iv => the giraffe dreams

- If we look at the example parse tree for the sample sentence in the illustration the giraffe dreams, the graphical illustration shows the parse tree for the sentence
- We can see that the root of every subtree has a grammatical category that appears on the left-hand side of a rule, and the children of that root are identical to the elements on the right-hand side of that rule.

Classification of Symbols in CFG

The symbols used in Context-free grammar are divided into two classes.

- The symbols that correspond to words in the language, for example, the nightclub, are called terminal symbols, and the lexicon is the set of rules that introduce these terminal symbols.
- The symbols that express abstractions over these terminals are called non-terminals.
- In each context-free rule, the item to the right of the arrow (→) is an ordered list of one or more terminals and non-terminals, and to the left of the arrow is a single non-terminal symbol expressing some cluster or generalization. - The non-terminal associated with each word in the lexicon is its lexical category or part of speech.

Context Free Grammar consists of a finite set of grammar rules that have four components: a Set of Non-Terminals, a Set of Terminals, a Set of Productions, and a Start Symbol.

CFG can also be seen as a notation used for describing the languages, a superset of Regular grammar.

CFG consists of a finite set of grammar rules having the following four components

- **Set of Non-terminals:** It is represented by V. The non-terminals are syntactic variables that denote the sets of strings, which help define the language generated with the help of grammar.
- **Set of Terminals:** It is also known as tokens and is represented by Σ. Strings are formed with the help of the basic symbols of terminals.
- **Set of Productions:** It is represented by P. The set explains how the terminals and non-terminals can be combined.

Every production consists of the following components:

- Non-terminals are also called variables or placeholders as they stand for other symbols, either terminals or non-terminals. They are symbols representing the structure of the language being described. Non-terminals are a set of production rules specifying how to replace a non-terminal symbol with a string of symbols, which can include terminals, words or characters, and other non-terminals.
- **Start Symbol:** The formal language defined by a CFG is the set of strings derivable from the designated start symbol. Each grammar must have one designated start symbol, which is often called S.
- Since context-free grammar is often used to define sentences, S is usually interpreted as the sentence node, and the set of strings that are derivable from S is the set of sentences in some simplified version of English.

Issues with using context-free grammar in NLP:

- **Limited expressiveness:** Context-free grammar is a limited formalism that cannot capture certain linguistic phenomena such as idiomatic expressions, coordination and ellipsis, and even long-distance dependencies.
- **Handling idiomatic expressions:** CFG may also have a hard time handling idiomatic expressions or idioms, phrases whose meaning cannot be inferred from the meanings of the individual words that make up the phrase.
- **Handling coordination:** CFG needs help to handle coordination, which is linking phrases or clauses with a conjunction.
- **Handling ellipsis**: Context-free grammar may need help to handle ellipsis, which is the omission of one or more words from a sentence that is recoverable from the context.

The limitations of context-free grammar can be mitigated by using other formalisms such as dependency grammar which is powerful but more complex to implement, or using a hybrid approach where both constituency and dependency are used together. We can also additionally use machine learning techniques in certain cases.

Constituency Grammar

Constituency Grammar is also known as Phrase structure grammar. Furthermore, it is called constituency Grammar as it is based on the constituency relation. It is the opposite of dependency grammar.

- The constituents can be any word, group of words or phrases in Constituency Grammar. The goal of constituency grammar is to organize any sentence into its constituents using their properties.
- Characteristic properties of constituency grammar and constituency relation:
    - All the related frameworks view the sentence structure in terms of constituency relation.

  - To derive the constituency relation, we take the help of subject-predicate division of Latin as well as Greek grammar.
  - In constituency grammar, we study the clause structure in terms of noun phrase NP and verb phrase VP.
- The properties are derived generally with the help of other NLP concepts like part of speech tagging, a noun or Verb phrase identification, etc. For example, Constituency grammar can organize any sentence into its three constituents - a subject, a context, and an object.

Look at a sample parse tree: Example sentence - "The dog chased the cat."

- In this parse tree, the sentence is represented by the root node S (for sentence). The sentence is divided into two main constituents: NP (noun phrase) and VP (verb phrase).
- The NP is further broken down into Det (determiner) and Noun, and the VP is further broken down into V (verb) and NP.
- Each of these constituents can be further broken down into smaller constituents.

Constituency grammar is better equipped to handle context-free and dependency grammar limitations. Let us look at them:

- Constituency grammar is not language-specific, making it easy to use the same model for multiple languages or switch between languages, hence handling the multilingual issue plaguing the other two types of grammar.
- Since constituency grammar uses a parse tree to represent the hierarchical relationship between the constituents of a sentence, it can be easily understood by humans and is more intuitive than other representation grammars.
- Constituency grammar also is simple and easier to implement than other formalisms, such as dependency grammar, making it more accessible for researchers and practitioners.
- Constituency grammar is robust to errors and can handle noisy or incomplete data.
- Constituency grammar is also better equipped to handle coordination which is the linking of phrases or clauses with a conjunction.

Dependency Grammar

Dependency Grammar is the opposite of constituency grammar and is based on the dependency relation. It is opposite to the constituency grammar because it lacks phrasal nodes.

Let us look at some fundamental points about Dependency grammar and dependency relation.

- Dependency Grammar states that words of a sentence are dependent upon other words of the sentence. These Words are connected by directed links in dependency grammar. The verb is considered the center of the clause structure.

- Dependency Grammar organizes the words of a sentence according to their dependencies. Every other syntactic unit is connected to the verb in terms of a directed link. These syntactic units are called dependencies.
    - One of the words in a sentence behaves as a root, and all the other words except that word itself are linked directly or indirectly with the root using their dependencies.
    - These dependencies represent relationships among the words in a sentence, and dependency grammar is used to infer the structure and semantic dependencies between the words.

Dependency grammar suffers from some limitations; let us understand them further.

- **Ambiguity:** Dependency grammar has issues with ambiguity when it comes to interpreting the grammatical relationships between words, which are particularly challenging when dealing with languages that have rich inflections or complex word order variations.
- **Data annotation:** Dependency parsing also requires labeled data to train the model, which is time-consuming and difficult to obtain.
- **Handling long-distance dependencies:** Dependency parsing also has issues with handling long-term dependencies in some cases where the relationships between words in a sentence may be very far apart, making it difficult to accurately capture the grammatical structure of the sentence.
- **Handling ellipsis and coordination:** Dependency grammar also has a hard time handling phenomena that are not captured by the direct relationships between words, such as ellipsis and coordination, which are typically captured by constituency grammar.

The limitations of dependency grammar can be mitigated by using constituency grammar which, although less powerful, but more intuitive and easier to implement. We can also use a hybrid approach where both constituency and dependency are used together, and it can be beneficial.

- Comparing Constituency grammar with Dependency grammar: Constituency grammar focuses on grouping words into phrases and clauses, while dependency grammar focuses on the relationships between individual words. Each word in a sentence is represented by a node in a dependency graph, and the edges between nodes represent the grammatical relationships between the words.
    - Dependency grammar is typically more powerful for some languages and NLP tasks as it captures the relationships between the words more accurately, but also more complex to implement and less intuitive.
    - Constituency grammar is more intuitive and easier to implement but can be less expressive.

Probabilistic Parsing

How can we choose the correct tree for a given sentence? Traditional approach: grammar rules hand-written by linguists

- constraints added to limit unlikely parses for sentences
- hand-written grammars are not robust: often fail to parse new sentences.

Current approach: use probabilities

- Probabilitistic CFG (PCFG) a CFG where each rule is augmented with a probability

$S \rightarrow NP\ VP$	.8
$S \rightarrow VP$	.2
$NP \rightarrow D\ N$	.4
$NP \rightarrow NP\ PP$	.4
$NP \rightarrow PN$	.2
$VP \rightarrow V\ NP$	.7
$VP \rightarrow VP\ PP$	.3
$PP \rightarrow P\ NP$	1

$D \rightarrow the$	.8
$D \rightarrow a$	.2
$N \rightarrow flight$	1
$PN \rightarrow john$	.9
$PN \rightarrow schiphol$	.1
$V \rightarrow booked$	1
$P \rightarrow from$	1

Using PCFGs, the probability of a parse tree for a given sentence is then given by the product of the probabilities of all the grammar rules used in the sentence derivation.

These probabilities can provide a criterion for disambiguation:

- i.e. a ranking over possible parses for any sentence
- we can choose the parse tree with the highest probability.

Treebank PCFGs

- Treebanks: instead of paying linguists to write a grammar, pay them to annotate real sentences with parse trees.
- This way, we implicitly get a grammar (for CFG: read the rules off the trees)
- And we get probabilities for those rules
- We can use these probabilities to improve disambiguation and also speed up parsing.

Estimating rule probabilities from a treebank

An estimated probability of a rule can be obtained by maximum likelihood estimate:

Here, numerator is the number of times the rule is used in the corpus, and denominator is the number of times the nonterminal X appears in the treebank.

UNIT-2 Word Level and Syntactic Analysis

Word Level Analysis Machine-readable dictionaries and lexical databases

LECTURE NO:- 5

MRD is a machine readable dictionary which consists of lexical databases which groups English words into sets of synonyms called Synsets, which gives simple meanings of the word. It maintains various semantic relations among synonym sets.

- It maintains rich vocabulary organizational structure. Using this it is very easy to construct and expand a domain lexicon. It provides rich semantic relations of words including synonym, antonym, and so on with which words are linked together to form a network. It is a structural notion where the meaning of a concept determined using its position relative to the other words in the structure.

- It is a dictionary in an electronic form that can be loaded in a database and can be queried via application software. It may be a single language explanatory dictionary or a multi-language dictionary to support translations between two or more languages or a combination of both.

- An MRD may be a dictionary with a proprietary structure that is queried by dedicated software (for example online via internet) or it can be a dictionary that has an open structure and is available for loading in computer databases and thus can be used via various software applications.

II. NEED FOR MRD'S Conventional dictionaries contain a lemma with various descriptions. A machine-readable dictionary may have additional capabilities and is therefore sometimes called a Smart Dictionary. An example of a smart dictionary is the Open Source Gellish English dictionary. Machine-readable versions of everyday dictionaries have been seen as a likely source of information for use in natural language processing because they contain an enormous amount of lexical and semantic knowledge. Machine-readable versions of everyday dictionaries have been seen as a likely source of information for use in NLP because they contain an enormous amount of lexical and semantic knowledge collected together over years of effort by lexicographers. Interest in machine-readable dictionaries (MRDs) as a ready-made source of knowledge has waned somewhat in recent years. Considerable research has been devoted to devising methods to extract this information from dictionaries based on the supposition that it is sufficient to form the kernel of a knowledge base that can be extended by utilizing information from other sources.

IV. STRUCTURE OF MRD

The lexical database for English language acts as a machine-readable dictionary, created and maintained by George Miller at the Cognitive Science Laboratory at Princeton University. It is an online lexical database designed for use under program control [1].

These database group English words into sets of synonyms called synsets, provides short, general definitions, and records the various semantic relations between these synonym sets.

Each such synset represents a single distinct sense or concept. For example, in MRD, the synset {car, auto, automobile, machine, motorcar} represents the concept of ―4-wheeled motor vehicle.

There are various semantic relations among synonym sets which serve as a combination of dictionary and thesaurus for automated text analysis. MRD stores information about words belong to four parts–of–speech: nouns, verbs, adjectives and adverbs.MRD‘s database groups English nouns, verbs, adjectives, and adverbs into synsets that are in turn linked through semantic relations that determine word definitions and senses.

- Organized around the notion of synsets (sets of synonymous words) Groups words together based on their meanings. It interlinks specific senses of words.
- Basic semantic relations between these synsets that labels the semantic relations among words.
- Noun hierarchy partitioned into separate hierarchies with unique top hypernyms.
- Neighbouring words will be more closely related to correct sense Words that are found in close proximity to one another in the network are semantically disambiguated.

MRD is an electronic lexical database, which is arranged semantically and contains nouns, verbs, adjectives and adverbs. Words that are synonymous are grouped together in Synonym sets are referred as synsets. Each synset has an associated definition named gloss, which is a short explanation of the meaning of the concept represented by the synset.

Many words in MRD are polysemous i.e. they are included in more than one synsets. For example, the word computer can be found in the synset {computer, computing machine, computing device, data processor, electronic computer, information processing system}, which has the gloss ―a machine for performing calculations automatically‖, and in the synset {calculator, reckoner, figurer, estimator, computer} which has the gloss ―an expert at calculation (or at operating calculating machines)‖.

Synsets are connected to each other through various semantic relations. The most important relations between nouns are the relations of hyponymy and hypernymy, which are transitive relations between synsets. The hypernymy relationship between synsets A and B means that B is a kind of A. Hypernymy and hyponymy are inverse relationships, so if A is a hypernym of B, then B is a hyponym of A.

For example the synset {computer, computing machine, computing device, data processor, electronic computer, information processing system} is a hypernym of the synset {home computer}. Usually each synset has only one hypernym, therefore this relation organizes WordNet into a hierarchical structure. Another pair of inverse relations that hold between nouns is the meronymy and the holonymy relations. If A is a holonym of B (or in other words B is a meronym of A), it means that B is a part of A. For example, synset {keyboard} is a meronym of the synset {computer, computing machine, computing device, data processor, electronic computer, information processing system [1].

UNIT-2 Word Level and Syntactic Analysis

Syntactic Analysis RTN, ATN

LECTURE NO:-6

TYPES OF TRANSITION NETWORK

There are generally two types of transition networks like

1. Recursive Transition networks (RTN)
2. Augmented Transition networks (ATN)

Let us focus on these two transition networks and their structure for parsing a sentence.

1. Recursive Transition Networks (RTN)

RTNs are considered as development for finite state automata with some essential conditions to take the recursive complexion for some definitions in consideration. A recursive transition network consists of nodes (states) and labeled arcs (transitions). It permits arc labels to refer to other networks and they in turn may refer back to the referring network rather than just permitting word categories. It is a modified version of transition network. It allows arc labels that refer to other networks rather than word category. A recursive transition network can have 5 types of arcs (Allen's, JM's) like

1) **CAT:** Current word must belong to category.
2) **WORD:** Current word must match label exactly.
3) **PUSH:** Named network must be successfully traversed.
4) **JUMP:** Can always be traversed.
5) **POP:** Can always be traversed and indicates that input string has been accepted by the network. In RTN, one state is specified as a start state. A string is accepted by an RTN if a POP arc is reached and all the input has been consumed. Let us consider a sentence "The stone was dark black".

Here **The:** ART

Stone: ADJ NOUN

Was: VERB

Dark: ADJ

Black: ADJ NOUN

The RTN structure is given in figure

The RTN structure is given in figure

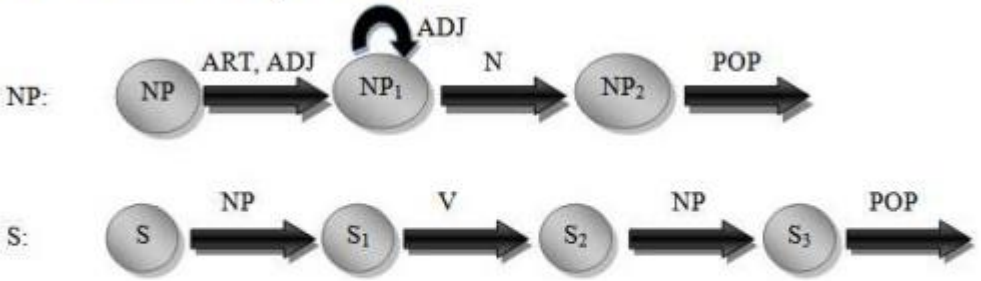


Figure RTN Structure

So we can parse the sentence through the RTN structure as follows.

Current	Word	Return	Arc	Backup States
S	1	-------	NP	----------
NP	1	S_1	ART	----------
NP_1	2	S_1	NOUN	(NP_1, 3, S_1)
NP_2	3	S_1	POP	(NP_1, 3, S_1) (NP_2, 4, S_1)
S_1	3	-------	VERB	(NP_1, 3, S_1) (NP_2, 4, S_1)
S_2	4	-------	NP	(NP_1, 3, S_1) (NP_2, 4, S_1)
NP	4	S_3	NONE	(NP_1, 3, S_1) (NP_2, 4, S_1)
NP_1	3	S_1	NOUN	(NP_2, 4, S_1)
NP_2	4	S_1	POP	(NP_2, 4, S_1)
S_1	4	-------	VERB	(NP_2, 4, S_1)
S_2	5	-------	NP	(NP_2, 4, S_1)
NP	5	S_3	ADJ	(NP_2, 4, S_1)
NP_1	6	S_3	NOUN	(NP_2, 4, S_1) (NP_1, 7, S_3)
NP_2	7	S_3	POP	(NP_2, 4, S_1) (NP_1, 7, S_3)
NP_2	7	S_3	POP	(NP_2, 4, S_1) (NP_1, 7, S_3)
S_3	7	-------	POP	(NP_2, 4, S_1) (NP_1, 7, S_3)

Finally as there are no words left so the parse is successful.

Also there is an another structure of RTN is described by William Woods (1970) is illustrated in figure. He described the total RTN structure into three parts like sentence (S), Noun Phrase (NP), Preposition Phrase (PP).

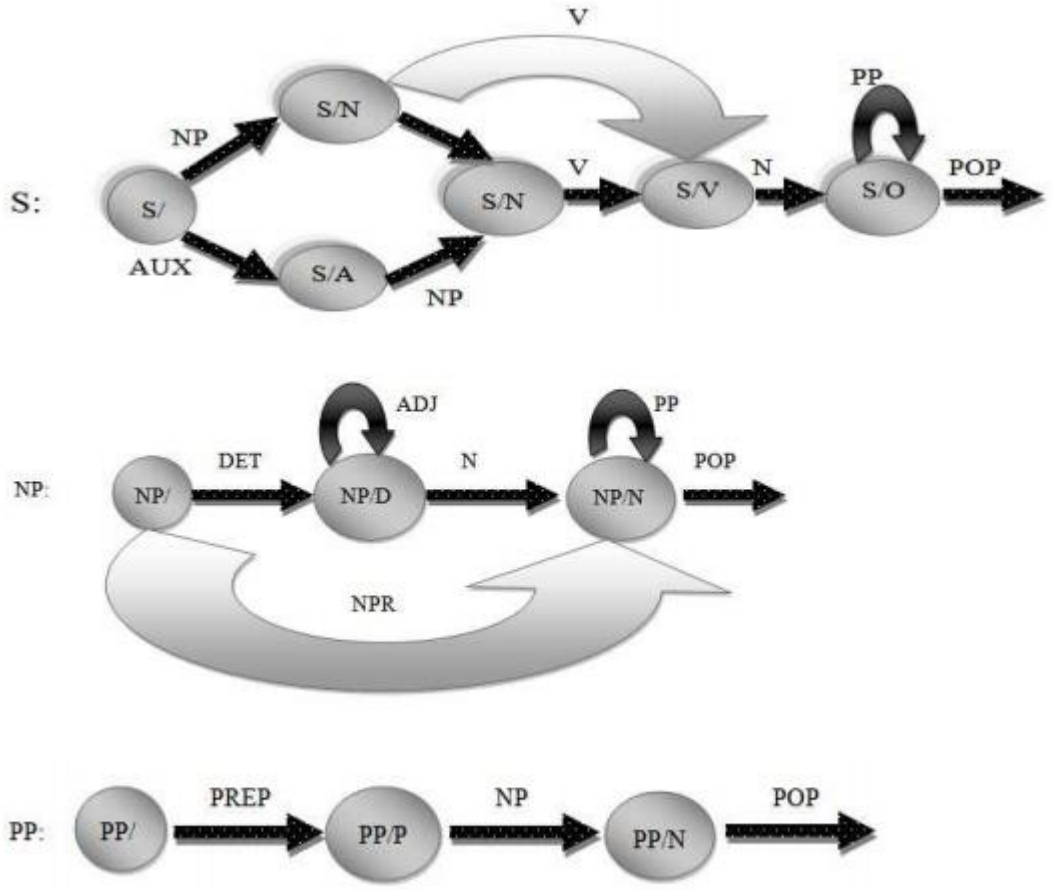


Figure RTN Structure

The number of sentences accepted by an RTN can be extended if backtracking is permitted when a failure occurs. This requires that states having alternative transitions be remembered until the parse progresses past possible failure points. In this way, if a failure occurs at some point, the interpreter can backtrack and try alternative paths. The disadvantage with this approach is that parts of a sentence may be parsed more than time resulting in excessive computations. During the traversal of an RTN, a record must be maintained of the word position in the input sentence and the current state and return nodes to be used as return points when control has been transformed to a lower level network.

2. Augmented Transition Network (ATN)

An ATN is a modified transition network. It is an extension of RTN. The ATN uses a top down parsing procedure to gather various types of information to be later used for understanding system. It produces the data structure suitable for further processing and capable of storing semantic details. An augmented transition network (ATN) is a recursive transition network that can perform tests and take actions during arc transitions. An ATN uses a set of registers to store information. A set of actions is defined for each arc and the actions can look at and modify the registers. An arc may have a test associated with it. The arc is traversed (and its action) is taken only if the test succeeds. When a lexical arc is traversed, it is put in a special variable (*) that keeps track of the current word. The ATN was first used in LUNAR system. In ATN, the arc can have a further arbitrary test and an arbitrary action. The structure of ATN is illustrated in figure. Like RTN, the structure of ATN is also consisting of the substructures of S, NP and PP.

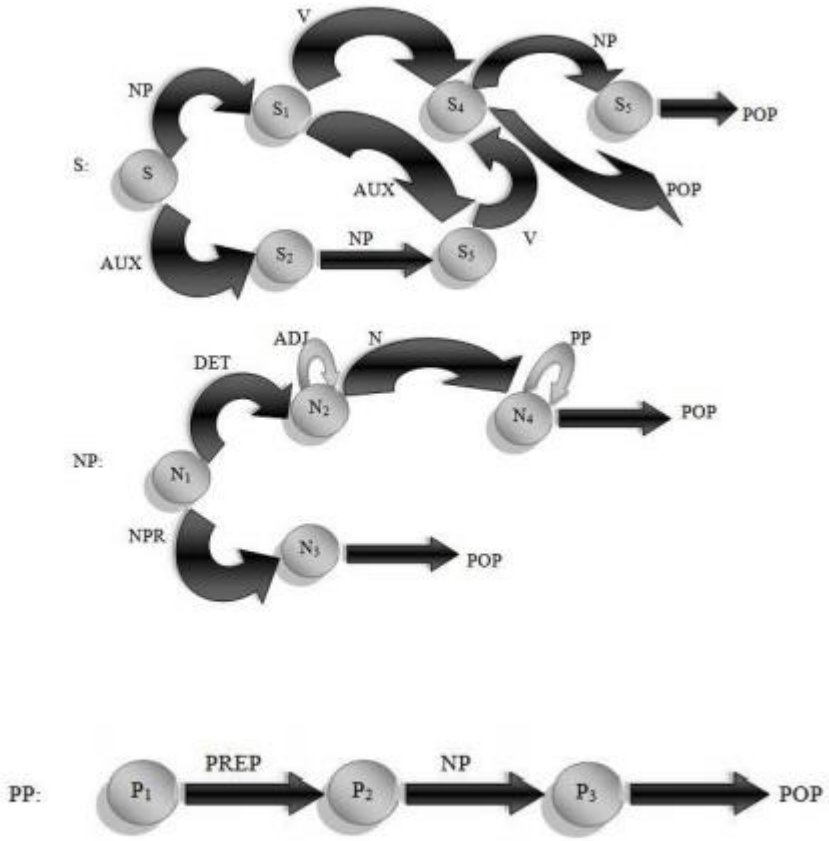


Figure ATN Structure

The ATN collects the sentence features for further analysis. The additional features that can be captured by the ATN are; subject NP, the object NP, the subject verb agreement, the declarative or interrogative mood, tense and so on. So we can conclude that ATN requires some more analysis steps compared to that of RTN. If these extra analysis tests are not performed, then

there must some ambiguity in ATN. The ATN represents sentence structure by using a slot filter representation, which reflects more of the functional role of phrases in a sentence. For example, one noun phrase may be identified as "subject" (SUBJ) and another as the "object" of the verb. Wit hin noun phrases, parsing will also identify the determiner structure, adjectives, the noun etc. For the sentence "Ram ate an apple", we can represent as in figure.

```
(S  SUBJ (NP   NAME    Ram)
         MAIN_V   ate
         TENSE    PAST
         OBJ (NP  DET    an
         HEAD     apple))
```

Figure Representation of sentence in ATN

The ATN maintains the information by having various registers like DET, ADJ and HEAD etc. Registers are set by actions that can be specified on the arcs. When the arc is followed, the specified action associated with it is executed. An ATN can recognize any language that a general purpose computer can recognize. The ATNs have been used successfully in a number of natural language systems as well as front ends for databases and expert systems.